

Sistemas Operacionais

Aula 2: Kernel

Prof. Me. Emerson Ap. Mouco Jr.

Apagando...

O que aconteceria se um programa pudesse enviar um comando de 'desligar' diretamente para o processador ou apagar a memória de outro programa?

Quem está no controle?

Como o processador sabe quem exatamente está no controle?

Bit de modo

- Os processadores modernos possuem um bit de status (Bit de Modo) que define o nível de privilégio atual.
- Modo usuário (User mode) e Modo kernel (Supervisor mode)

Bit de modo

- **Modo usuário (User mode):** Modo restrito. Onde rodam o navegador, o Word, os jogos. Instruções perigosas (como acessar o disco rígido diretamente) são proibidas
- **Modo kernel (Supervisor mode):** Acesso total e irrestrito ao hardware. Onde o "coração" do Sistema Operacional roda

Bit de modo

- O "Modo usuário" é a área de atendimento de um banco (você falar com o caixa etc.)
- O "Modo kernel" é o cofre do banco. Clientes não entram no cofre; eles pedem ao funcionário (o SO) para buscar o dinheiro

Bit de modo

- Se algum programa de "Modo usuário" tentar executar uma tarefa no processador, gera uma execução/um chamado para o SO matar processo

System call

- Se um programa de "Modo usuário" não pode executar tarefas, como ele pode "salvar um arquivo" em disco?
- Ele usa uma "chamada de sistema/system call"

O passo a passo

- 1- O programa comum (de modo usuário) diz "Preciso escrever algo na tela!" (ex: `printf()` em C). Mas ele é apenas um software comum e não tem permissão para mexer no hardware
- 2- A biblioteca C age como um auxiliar. Ela coloca o código do pedido (ex: `sys_write = 1`) em uma caixinha de correio específica chamada Registrador (ex: EAX)

O passo a passo

Obs: O EAX (Extended Accumulator Register) é um registrador de 32 bits fundamental na arquitetura x86 (Intel/AMD), usado principalmente para armazenar valores temporários, resultados de operações aritméticas e lógicas, e operandos em instruções de linguagem Assembly

O passo a passo

- 3- O programa executa a instrução especial TRAP ou INT 0x80. É como se apertássemos um botão de pânico ou a campainha do balcão (Interrupção de Software)
- 4- O hardware ouve esse chamado, muda a chave do sistema para o "modo segurança" e chama o SO. O SO então analisa a Tabela de Vetores de Interrupção para saber quem resolve o pedido #1 que está na caixa EAX

O passo a passo

5- O SO executa a tarefa com total segurança, salva no disco, muda a chave de volta para o Modo usuário e avisa ao programa: "Pronto, serviço feito, pode continuar sua vida!"

O passo a passo

- Mudar de Usuário para Kernel é "caro" em termos de processamento. Por isso, os programadores devem otimizar o código para não fazer milhares de system calls desnecessárias

System calls

- Imagine que o Modo usuário é a sua casa e o Modo kernel (onde fica o disco rígido) é o supermercado
- A System Call é a viagem de carro da sua casa até o supermercado

System calls

- **O Programador Ruim (Código não otimizado):** Ele quer fazer um risoto. Ele dirige até o supermercado (System Call), compra um único grão de arroz, volta para casa e coloca na panela
- Depois, ele dirige de novo até o mercado, compra o segundo grão de arroz, e volta. Ele faz 10.000 viagens. O problema não é o peso do arroz, mas o tempo gasto no trânsito (a mudança de contexto entre usuário e kernel)

System calls

- **O Programador Bom (Código otimizado - uso de Buffer):** Ele pega um carrinho de compras grande (um Buffer na memória RAM), dirige até o mercado uma única vez (1 System Call), pega um saco de arroz, e volta para casa

System calls

Se um aluno for criar um programa em C ou Java para ler um arquivo de texto de 1 Megabyte (1 milhão de caracteres)

System calls

- Se ele usar o comando de leitura para ler letra por letra (1 byte por vez), o programa fará 1.000.000 de System Calls
- A CPU vai parar o que está fazendo, mudar para o Modo kernel, ler 1 letra, voltar para o Modo usuário e repetir isso um milhão de vezes, tornando o programa extremamente lento

System calls

- Se o aluno criar um Array/Buffer de 1 Megabyte e pedir para o SO: "Leia o arquivo inteiro de uma vez e jogue aqui dentro", o programa fará apenas 1 System Call
- O SO vai ao disco, pega tudo, devolve para o programa, e o sistema funciona otimizado

Kernel

O que é o kernel?



Kernel

- É a parte central do SO que fica residente na memória RAM o tempo todo
- Kernel monolítico e Microkernel

Kernel monolítico

- O SO inteiro (gerência de processos, memória, sistema de arquivos, drivers de vídeo e rede etc.), ele é um único grande programa rodando no Modo kernel
- **Vantagem:** Alta performance, as funções chamam umas às outras diretamente

Kernel monolítico

- **Desvantagem:** Baixa confiabilidade, um bug em um driver de webcam de terceiros que roda no kernel vai derrubar o SO inteiro (O famoso Kernel Panic ou Tela Azul)
- Exemplo: Linux, MS-DOS etc.

Microkernel

- O Kernel é reduzido ao mínimo absoluto (apenas comunicação e escalonamento)
- Todo o resto (drivers, sistema de arquivos) roda como processos comuns no Modo usuário
- **Vantagem:** Isolamento e Segurança, se o driver de vídeo falha, apenas ele reinicia. O SO não trava

Microkernel

- **Desvantagem:** Muito lento, para um programa ler o disco, ele pede ao Kernel, que repassa a mensagem para o "Servidor de Arquivos", que repassa pro Kernel, que fala com o disco...
- Exemplo: MINIX, QNX (usado em sistemas automotivos críticos)

Linus Torvalds vs. Andrew Tanenbaum

- Tanenbaum (criador do MINIX) disse que o Linux era obsoleto por nascer Monolítico nos anos 90, argumentando que o futuro era o Microkernel
- Torvalds defendeu o Linux dizendo que Microkernels eram teóricos demais e muito lentos na prática.

Linus Torvalds vs. Andrew Tanenbaum

- **Quem ganhou?**
- O Linux domina os servidores/celulares hoje, mas os microkernels dominam áreas de tempo real e segurança crítica

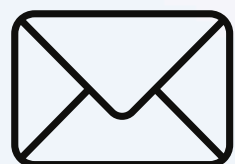
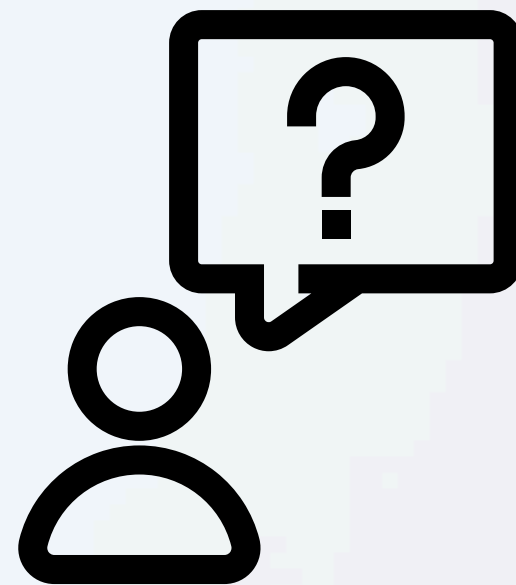
Windows e MacOS

- Rodam kernel híbrido, ou seja, eles são construídos com a organização de um microkernel (o código é dividido em módulos elegantes e independentes)
- No entanto, na hora de o sistema executar, eles carregam quase todos esses módulos juntos para dentro do Modo Kernel

Windows e MacOS

- **Vantagem:** a performance, ganha-se a velocidade de um sistema Monolítico, pois os componentes não perdem tempo enviando mensagens uns aos outros "de fora para dentro do cofre", tudo processa rapidamente
- **Desvantagem:** a fragilidade, herda-se o grande problema do Monolítico. Como tudo está rodando com privilégios máximos, se um único módulo falhar (ex: um driver de placa de vídeo), ele corrompe a memória do Kernel e o sistema inteiro é forçado a desligar para se proteger (tela azul no Windows)

Dúvidas?



emerson.mouco@fatec.sp.gov.br



17 982236363